

1. What is an Algorithm?

- **Definition:** An algorithm is a finite set of instructions, commands, or a step-by-step procedure carried out in a specific order to solve logical, mathematical, or real-world problems.
- **Real-World Analogy:** A recipe is an example of an algorithm. It outlines step-by-step instructions, taking inputs (ingredients) to produce an output (the finished meal).
- **Core Components:**
 - **Problem:** A real-world instance or issue requiring a program or set of instructions.
 - **Input:** The necessary and desired values provided to the algorithm.
 - **Processing Unit:** The component (like the CPU) that processes the input to create the desired output.
 - **Output:** The final outcome or result produced by the program.

2. Characteristics of an Algorithm

To be effective, an algorithm must possess the following traits:

- **Input:** Requires some input values (which can be given a value other than 0).
- **Output:** Will result in one or more outcomes at the end.
- **Unambiguity:** Instructions must be perfectly clear, straightforward, and unambiguous.
- **Finiteness:** It must have a limited, countable number of instructions (it must end).
- **Effectiveness:** Every instruction affects the overall process and must be adequate.
- **Language Independence:** Instructions must be generic enough to be implemented in any programming language and still produce the exact same results.

3. Programming Constructs (The 3 Basic Building Blocks)

Programs are built using three fundamental structures that dictate how code executes:

I. Sequence

- **Meaning:** The most basic construct where instructions occur and execute one after another in order.
- **Process:** The computer runs code line-by-line, from top to bottom (e.g., Line 1, Line 2, Line 3), until it reaches the end.
- **Example:** A school child's daily schedule (Wake up, Brush teeth, Wash, Dress, Eat breakfast, etc.).

II. Selection

- **Meaning:** Determines which path a program takes depending on whether a specific condition is met.
- **Process:** If a condition is true, certain lines of code execute. If false, the computer ignores/skips those lines and jumps ahead.
- **Implementation:** Achieved using **IF statements**.

III. Iteration (Looping)

- **Meaning:** The repeated execution of a specific section of code while a program is running.
- **Process:** Instead of rewriting the same lines of code multiple times, a **loop** forces the program to repeat a set of steps as many times as required.

4. Key Computational Concepts

- **Abstraction:** The process of hiding background details or unnecessary implementation complexities so users only see the required information.
 - *Data Abstraction Example:* When using a washing machine, you simply put in clothes and wait. You do not need to understand the complex engineering or mechanisms operating inside the machine.
- **Decomposition:** The process of breaking down a large, complex problem into smaller, manageable parts.
 - *Why it matters:* Smaller parts are easier to examine, design, and solve individually. Without decomposition, problems are much harder for a computer's CPU to process efficiently.

5. Practical Examples & Applications

A. Simple Multiplication Algorithm

A step-by-step plan to multiply two numbers and display the result:

- **Step 1** – Start
- **Step 2** – Declare three integers: x, y, and z
- **Step 3** – Define the values of x and y

- **Step 4** – Multiply the values of x and y
- **Step 5** – Store the result of Step 4 into z
- **Step 6** – Print z
- **Step 7** – End

B. Linear Search Algorithm

- **Definition:** The simplest sequential searching method, where you start at one end of a list and check every single element one by one until the desired item (the "key") is found.
- **How it works:**
 1. Every element in an array is treated as a potential match.
 2. If an element equals the key, the search is successful and its index location is returned.
 3. If the loop checks every item and finds nothing, it returns "No match found".
- **Visual Walkthrough:**
 - Given array: `arr[] = {10, 50, 30, 70, 80, 20, 90, 40}` and target **Key = 30**
 - *Step 1:* Compare Key (30) with `arr[0]` (10) Not equal Move to next element.
 - *Step 2:* Compare Key (30) with `arr[1]` (50) Not equal Move to next element.
 - *Step 3:* Compare Key (30) with `arr[2]` (30) **Match found!** Algorithm yields success and returns index 2.